



Kubernetes config files

How to create objects

Use `kubectl apply` to create all objects, except those that already exist, defined by configuration files in a specified directory:

```
kubectl apply -f <directory>/
```

This sets the `kubectl.kubernetes.io/last-applied-configuration: '{...}'` annotation on each object. The annotation contains the contents of the object configuration file that was used to create the object.

Here's an example of an object configuration file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

How to create objects

Run `kubectl diff` to print the object that will be created:

```
kubectl diff -f https://k8s.io/examples/application/simple_deployment.yaml
```

Create the object using `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Print the live configuration using `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

How to create objects

The output shows that the `kubectl.kubernetes.io/last-applied-configuration` annotation was written to the live configuration, and it matches the configuration file:

```
kind: Deployment
metadata:
  annotations:
    # ...
    # This is the json representation of simple_deployment.yaml
    # It was written by kubectl apply when the object was created
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"minReadySeconds":5,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}},
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
```

How to update objects

You can also use `kubectl apply` to update all objects defined in a directory, even if those objects already exist. This approach accomplishes the following:

1. Sets fields that appear in the configuration file in the live configuration.
2. Clears fields removed from the configuration file in the live configuration.

```
kubectl diff -f <directory>/  
kubectl apply -f <directory>/
```

Here's an example configuration file:



```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-deployment  
spec:  
  selector:  
    matchLabels:  
      app: nginx  
  minReadySeconds: 5  
  template:  
    metadata:  
      labels:  
        app: nginx  
    spec:  
      containers:  
      - name: nginx  
        image: nginx:1.14.2  
        ports:  
        - containerPort: 80
```

How to update objects

Create the object using `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Print the live configuration using `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

How to update objects

The output shows that the `kubectl.kubernetes.io/last-applied-configuration` annotation was written to the live configuration, and it matches the configuration file:

```
kind: Deployment
metadata:
  annotations:
    # ...
    # This is the json representation of simple_deployment.yaml
    # It was written by kubectl apply when the object was created
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion": "apps/v1", "kind": "Deployment",
       "metadata": {"annotations": {}, "name": "nginx-deployment", "namespace": "default"},
       "spec": {"minReadySeconds": 5, "selector": {"matchLabels": {"app": "nginx"}}, "template": {"metadata": {"labels": {"app": "nginx"}},
       "spec": {"containers": [{"image": "nginx:1.14.2", "name": "nginx",
       "ports": [{"containerPort": 80}]}]}}}}
    # ...
spec:
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
```

How to update objects

Directly update the `replicas` field in the live configuration by using `kubectl scale`. This does not use `kubectl apply`:

```
kubectl scale deployment/nginx-deployment --replicas=2
```

Print the live configuration using `kubectl get`:

```
kubectl get deployment nginx-deployment -o yaml
```


How to update objects

The output shows that the `replicas` field has been set to 2, and the `last-applied-configuration` annotation does not contain a `replicas` field:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # note that the annotation does not contain replicas
    # because it was not updated through apply
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"minReadySeconds":5,"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}},
      "spec":{"containers":[{"image":"nginx:1.14.2","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  replicas: 2 # written by scale
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
```

How to update objects

Update the `simple_deployment.yaml` configuration file to change the image from `nginx:1.14.2` to `nginx:1.16.1`, and delete the `minReadySeconds` field:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.16.1 # update the image
          ports:
            - containerPort: 80
```

How to update objects

Apply the changes made to the configuration file:

```
kubectl diff -f https://k8s.io/examples/application/update_deployment.yaml  
kubectl apply -f https://k8s.io/examples/application/update_deployment.yaml
```

Print the live configuration using `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/update_deployment.yaml -o yaml
```

The output shows the following changes to the live configuration:

- The `replicas` field retains the value of 2 set by `kubectl scale`. This is possible because it is omitted from the configuration file.
- The `image` field has been updated to `nginx:1.16.1` from `nginx:1.14.2`.
- The `last-applied-configuration` annotation has been updated with the new image.
- The `minReadySeconds` field has been cleared.
- The `last-applied-configuration` annotation no longer contains the `minReadySeconds` field.

How to update objects

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # The annotation contains the updated image to nginx 1.11.9,
    # but does not contain the updated replicas to 2
    kubectrl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
       "metadata":{"annotations":{"name":"nginx-deployment"},"namespace":"default"},
       "spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}}},
        "spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx",
        "ports":[{"containerPort":80}]]}}}}}}
    # ...
spec:
  replicas: 2 # Set by 'kubectrl scale'. Ignored by 'kubectrl apply'.
  # minReadySeconds cleared by 'kubectrl apply'
  # ...
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.16.1 # Set by 'kubectrl apply'
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
  # ...
```

How to delete objects

There are two approaches to delete objects managed by `kubectl apply`.

Recommended: `kubectl delete -f <filename>`

Manually deleting objects using the imperative command is the recommended approach, as it is more explicit about what is being deleted, and less likely to result in the user deleting something unintentionally:

```
kubectl delete -f <filename>
```

How to delete objects

Alternative: `kubectl apply -f <directory/> --prune -l your=label`

Only use this if you know what you are doing.

As an alternative to `kubectl delete`, you can use `kubectl apply` to identify objects to be deleted after their configuration files have been removed from the directory. Apply with `--prune` queries the API server for all objects matching a set of labels, and attempts to match the returned live object configurations against the object configuration files. If an object matches the query, and it does not have a configuration file in the directory, and it has a `last-applied-configuration` annotation, it is deleted.

```
kubectl apply -f <directory/> --prune -l <labels>
```

How to view an object

You can use `kubectl get` with `-o yaml` to view the configuration of a live object:

```
kubectl get -f <filename|url> -o yaml
```

How apply calculates differences and merges changes

When `kubectl apply` updates the live configuration for an object, it does so by sending a patch request to the API server. The patch defines updates scoped to specific fields of the live object configuration. The `kubectl apply` command calculates this patch request using the configuration file, the live configuration, and the `last-applied-configuration` annotation stored in the live configuration.

Merge patch calculation

The `kubectl apply` command writes the contents of the configuration file to the `kubectl.kubernetes.io/last-applied-configuration` annotation. This is used to identify fields that have been removed from the configuration file and need to be cleared from the live configuration. Here are the steps used to calculate which fields should be deleted or set:

1. Calculate the fields to delete. These are the fields present in `last-applied-configuration` and missing from the configuration file.
2. Calculate the fields to add or set. These are the fields present in the configuration file whose values don't match the live configuration.

How apply calculates differences and merges changes

Here's an example. Suppose this is the configuration file for a Deployment object:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.16.1 # update the image
          ports:
            - containerPort: 80
```

How apply calculates differences and merges changes

Also, suppose this is the live configuration for the same Deployment object:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # note that the annotation does not contain replicas
    # because it was not updated through apply
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion": "apps/v1", "kind": "Deployment",
       "metadata": {"annotations": {}, "name": "nginx-deployment", "namespace": "default"},
       "spec": {"minReadySeconds": 5, "selector": {"matchLabels": {"app": "nginx"}}, "template": {"metadata": {"labels": {"app": "nginx"}},
       "spec": {"containers": [{"image": "nginx:1.14.2", "name": "nginx",
       "ports": [{"containerPort": 80}]}]}}}}
    # ...
spec:
  replicas: 2 # written by scale
  # ...
  minReadySeconds: 5
  selector:
    matchLabels:
      # ...
      app: nginx
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.14.2
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
```

How apply calculates differences and merges changes

Here are the merge calculations that would be performed by `kubectl apply`:

1. Calculate the fields to delete by reading values from `last-applied-configuration` and comparing them to values in the configuration file. Clear fields explicitly set to null in the local object configuration file regardless of whether they appear in the last-applied-configuration. In this example, `minReadySeconds` appears in the `last-applied-configuration` annotation, but does not appear in the configuration file. **Action:** Clear `minReadySeconds` from the live configuration.
2. Calculate the fields to set by reading values from the configuration file and comparing them to values in the live configuration. In this example, the value of `image` in the configuration file does not match the value in the live configuration. **Action:** Set the value of `image` in the live configuration.
3. Set the `last-applied-configuration` annotation to match the value of the configuration file.
4. Merge the results from 1, 2, 3 into a single patch request to the API server.

How apply calculates differences and merges changes

Here is the live configuration that is the result of the merge:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    # ...
    # The annotation contains the updated image to nginx 1.11.9,
    # but does not contain the updated replicas to 2
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment",
      "metadata":{"annotations":{},"name":"nginx-deployment","namespace":"default"},
      "spec":{"selector":{"matchLabels":{"app":"nginx"},"template":{"metadata":{"labels":{"app":"nginx"}},
      "spec":{"containers":[{"image":"nginx:1.16.1","name":"nginx",
      "ports":[{"containerPort":80}]}}}}}}
    # ...
spec:
  selector:
    matchLabels:
      # ...
      app: nginx
  replicas: 2 # Set by 'kubectl scale'. Ignored by 'kubectl apply'
  # minReadySeconds cleared by 'kubectl apply'
  # ...
  template:
    metadata:
      # ...
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx:1.16.1 # Set by 'kubectl apply'
          # ...
          name: nginx
          ports:
            - containerPort: 80
          # ...
        # ...
      # ...
    # ...
```

How different types of fields are merged

How a particular field in a configuration file is merged with the live configuration depends on the type of the field. There are several types of fields:

- **primitive:** A field of type string, integer, or boolean. For example, `image` and `replicas` are primitive fields. **Action:** Replace.
- **map, also called object:** A field of type map or a complex type that contains subfields. For example, `labels`, `annotations`, `spec` and `metadata` are all maps. **Action:** Merge elements or subfields.
- **list:** A field containing a list of items that can be either primitive types or maps. For example, `containers`, `ports`, and `args` are lists. **Action:** Varies.

When `kubectl apply` updates a map or list field, it typically does not replace the entire field, but instead updates the individual subelements. For instance, when merging the `spec` on a Deployment, the entire `spec` is not replaced. Instead the subfields of `spec`, such as `replicas`, are compared and merged.

Merging changes to primitive fields

Primitive fields are replaced or cleared.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

Merging changes to map fields

Fields that represent maps are merged by comparing each of the subfields or elements of the map:

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

Merging changes for fields of type list

Merging changes to a list uses one of three strategies:

- Replace the list if all its elements are primitives.
- Merge individual elements in a list of complex elements.
- Merge a list of primitive elements.

The choice of strategy is made on a per-field basis.

Replace the list if all its elements are primitives

Treat the list the same as a primitive field. Replace or delete the entire list.

This preserves ordering.

Example: Use `kubectl apply` to update the `args` field of a Container in a Pod.

This sets the value of `args` in the live configuration to the value in the configuration file. Any `args` elements that had previously been added to the live configuration are lost. The order of the `args` elements defined in the configuration file is retained in the live configuration.

```
# last-applied-configuration value
args: ["a", "b"]

# configuration file value
args: ["a", "c"]

# live configuration
args: ["a", "b", "d"]

# result after merge
args: ["a", "c"]
```

Explanation: The merge used the configuration file value as the new list value.

Merge individual elements of a list of complex elements:

Treat the list as a map, and treat a specific field of each element as a key. Add, delete, or update individual elements. This does not preserve ordering.

This merge strategy uses a special tag on each field called a `patchMergeKey`. The `patchMergeKey` is defined for each field in the Kubernetes source code: **types.go** When merging a list of maps, the field specified as the `patchMergeKey` for a given element is used like a map key for that element.

Example: Use `kubectl apply` to update the `containers` field of a PodSpec. This merges the list as though it was a map where each element is keyed by name.

Merge individual elements of a list of complex elements:

Explanation:

- The container named "nginx-helper-a" was deleted because no container named "nginx-helper-a" appeared in the configuration file.
- The container named "nginx-helper-b" retained the changes to args in the live configuration. kubectl apply was able to identify that "nginx-helper-b" in the live configuration was the same "nginx-helper-b" as in the configuration file, even though their fields had different values (no args in the configuration file). This is because the patchMergeKey field value (name) was identical in both.
- The container named "nginx-helper-c" was added because no container with that name appeared in the live configuration, but one with that name appeared in the configuration file.
- The container named "nginx-helper-d" was retained because no element with that name appeared in the last-applied-configuration.

```
# last-applied-configuration value
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-a # key: nginx-helper-a; will be deleted in result
  image: helper:1.3
- name: nginx-helper-b # key: nginx-helper-b; will be retained
  image: helper:1.3

# configuration file value
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-b
  image: helper:1.3
- name: nginx-helper-c # key: nginx-helper-c; will be added in result
  image: helper:1.3

# live configuration
containers:
- name: nginx
  image: nginx:1.16
- name: nginx-helper-a
  image: helper:1.3
- name: nginx-helper-b
  image: helper:1.3
  args: ["run"] # Field will be retained
- name: nginx-helper-d # key: nginx-helper-d; will be retained
  image: helper:1.3

# result after merge
containers:
- name: nginx
  image: nginx:1.16
  # Element nginx-helper-a was deleted
- name: nginx-helper-b
  image: helper:1.3
  args: ["run"] # Field was retained
- name: nginx-helper-c # Element was added
  image: helper:1.3
- name: nginx-helper-d # Element was ignored
  image: helper:1.3
```

Default field values

The API server sets certain fields to default values in the live configuration if they are not specified when the object is created.

Here's a configuration file for a Deployment. The file does not specify `strategy:`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

Default field values

Create the object using `kubectl apply`:

```
kubectl apply -f https://k8s.io/examples/application/simple_deployment.yaml
```

Print the live configuration using `kubectl get`:

```
kubectl get -f https://k8s.io/examples/application/simple_deployment.yaml -o yaml
```

Default field values

The output shows that the API server set several fields to default values in the live configuration. These fields were not specified in the configuration file.

```
apiVersion: apps/v1
kind: Deployment
# ...
spec:
  selector:
    matchLabels:
      app: nginx
  minReadySeconds: 5
  replicas: 1 # defaulted by apiserver
  strategy:
    rollingUpdate: # defaulted by apiserver - derived from strategy.type
    maxSurge: 1
    maxUnavailable: 1
  type: RollingUpdate # defaulted by apiserver
  template:
    metadata:
      creationTimestamp: null
    labels:
      app: nginx
  spec:
    containers:
      - image: nginx:1.14.2
        imagePullPolicy: IfNotPresent # defaulted by apiserver
        name: nginx
        ports:
          - containerPort: 80
            protocol: TCP # defaulted by apiserver
        resources: {} # defaulted by apiserver
        terminationMessagePath: /dev/termination-log # defaulted by apiserver
      dnsPolicy: ClusterFirst # defaulted by apiserver
      restartPolicy: Always # defaulted by apiserver
      securityContext: {} # defaulted by apiserver
      terminationGracePeriodSeconds: 30 # defaulted by apiserver
# ...
```

Default field values

In a patch request, defaulted fields are not re-defaulted unless they are explicitly cleared as part of a patch request. This can cause unexpected behavior for fields that are defaulted based on the values of other fields. When the other fields are later changed, the values defaulted from them will not be updated unless they are explicitly cleared.

For this reason, it is recommended that certain fields defaulted by the server are explicitly defined in the configuration file, even if the desired values match the server defaults. This makes it easier to recognize conflicting values that will not be re-defaulted by the server.

```
# last-applied-configuration
spec:
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

```
# configuration file
spec:
  strategy:
    type: Recreate # updated value
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```



```
# live configuration
spec:
  strategy:
    type: RollingUpdate # defaulted value
    rollingUpdate: # defaulted value derived from type
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80

# result after merge - ERROR!
spec:
  strategy:
    type: Recreate # updated value: incompatible with rollingUpdate
    rollingUpdate: # defaulted value: incompatible with "type: Recreate"
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```



Default field values

Explanation:

1. The user creates a Deployment without defining `strategy.type`.
2. The server defaults `strategy.type` to `RollingUpdate` and defaults the `strategy.rollingUpdate` values.
3. The user changes `strategy.type` to `Recreate`. The `strategy.rollingUpdate` values remain at their defaulted values, though the server expects them to be cleared. If the `strategy.rollingUpdate` values had been defined initially in the configuration file, it would have been more clear that they needed to be deleted.
4. Apply fails because `strategy.rollingUpdate` is not cleared. The `strategy.rollingupdate` field cannot be defined with a `strategy.type` of `Recreate`.

Recommendation: These fields should be explicitly defined in the object configuration file:

- Selectors and PodTemplate labels on workloads, such as Deployment, StatefulSet, Job, DaemonSet, ReplicaSet, and ReplicationController
- Deployment rollout strategy

How to change ownership of a field between the configuration file and direct imperative writers

These are the only methods you should use to change an individual object field:

- Use `kubectl apply`.
- Write directly to the live configuration without modifying the configuration file: for example, use `kubectl scale`.

Changing the owner from a direct imperative writer to a configuration file

Add the field to the configuration file. For the field, discontinue direct updates to the live configuration that do not go through `kubectl apply`.

Changing the owner from a configuration file to a direct imperative writer

As of Kubernetes 1.5, changing ownership of a field from a configuration file to an imperative writer requires manual steps:

- Remove the field from the configuration file.
- Remove the field from the `kubectl.kubernetes.io/last-applied-configuration` annotation on the live object.

Changing management methods

Kubernetes objects should be managed using only one method at a time. Switching from one method to another is possible, but is a manual process.

Migrating from imperative command management to declarative object configuration

Migrating from imperative command management to declarative object configuration involves several manual steps:

1. Export the live object to a local configuration file:

```
kubectl get <kind>/<name> -o yaml > <kind>_<name>.yaml
```

1. Manually remove the status field from the configuration file.
2. Set the `kubectl.kubernetes.io/last-applied-configuration` annotation on the object:

```
kubectl replace --save-config -f <kind>_<name>.yaml
```

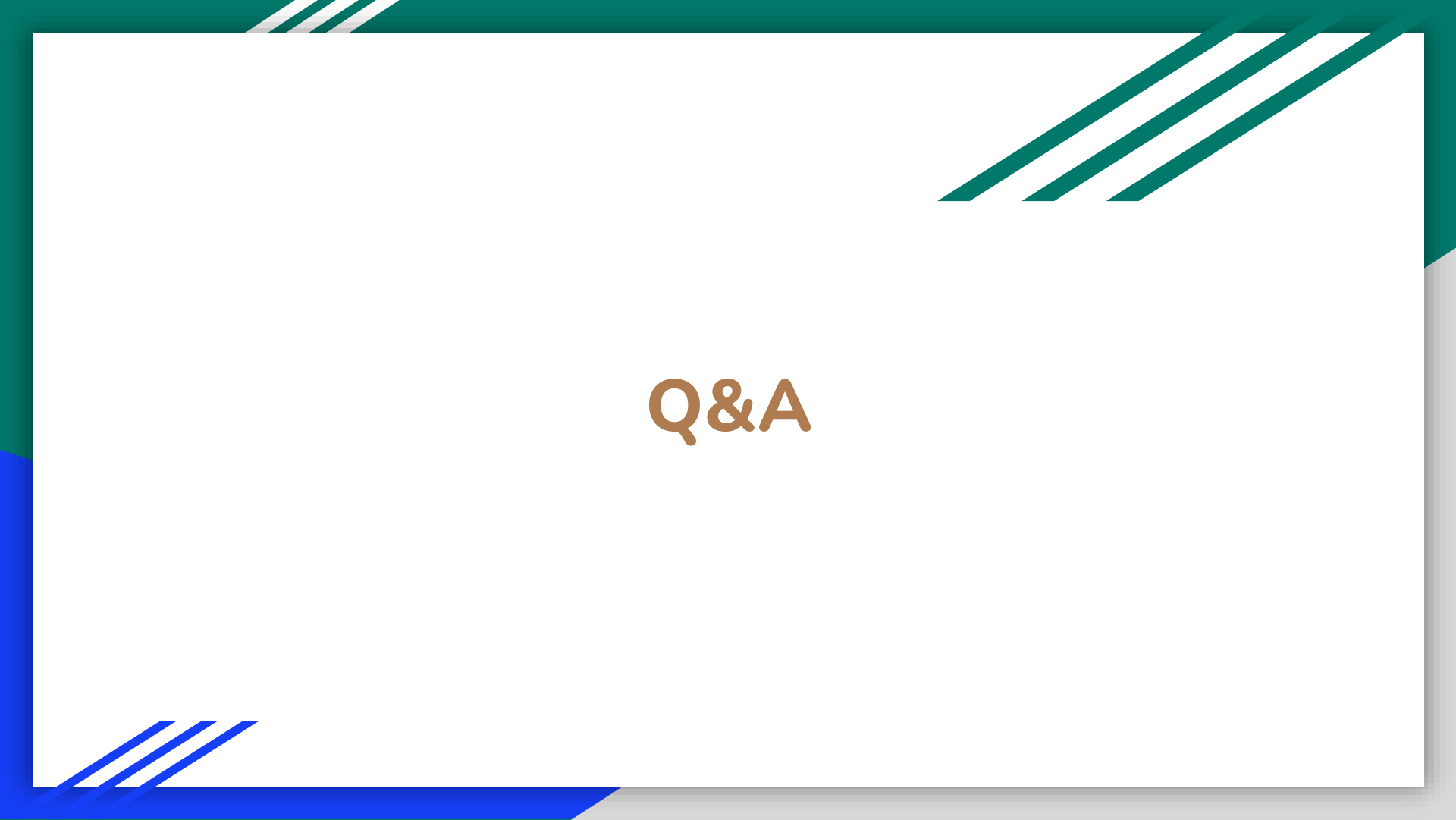
1. Change processes to use `kubectl apply` for managing the object exclusively.

Defining controller selectors and PodTemplate labels

The recommended approach is to define a single, immutable PodTemplate label used only by the controller selector with no other semantic meaning.

Example:

```
selector:
  matchLabels:
    controller-selector: "apps/v1/deployment/nginx"
template:
  metadata:
    labels:
      controller-selector: "apps/v1/deployment/nginx"
```



Q&A